Lab 7: Fuzzing for Input Validation Bugs

Pet Boarding (#49) — Property-based fuzz testing with Hypothesis

Student Name	Subramani B M
SRN	PES1UG24CS473
Section	H
Assigned Repository	https://github.com/SETAPESU26/L6_L7_49_pet_boarding
Solution Repository	https://github.com/SubramaniBM/L6_L7_49_pet_boarding (folder: Lab 7)
Environment	macOS, Python 3.15.2, pytest 9.2.0, Hypothesis 6.170.15

Objective

Use property-based fuzzing (Hypothesis) to find input-validation bugs in `pet_boarding.py`, fix them with defensive programming, and confirm the fixes by re-running the fuzz tests. The module has three functions: **parse_booking** (`pet:days`), **stay_average** (comma-separated stay lengths) and **payable_fee** (`fee|discount`, with 18% GST added).

Step 1: Install dependencies

```
pip install hypothesis
pip install pytest
```

Installed in a virtual environment named *lab7* (visible in the terminal prompt).

Step 2: Run the buggy application

Command: `python pet_boarding.py`. With well-formed input the program works. Typing just `dog` (no colon) crashes it with an unhandled ValueError.

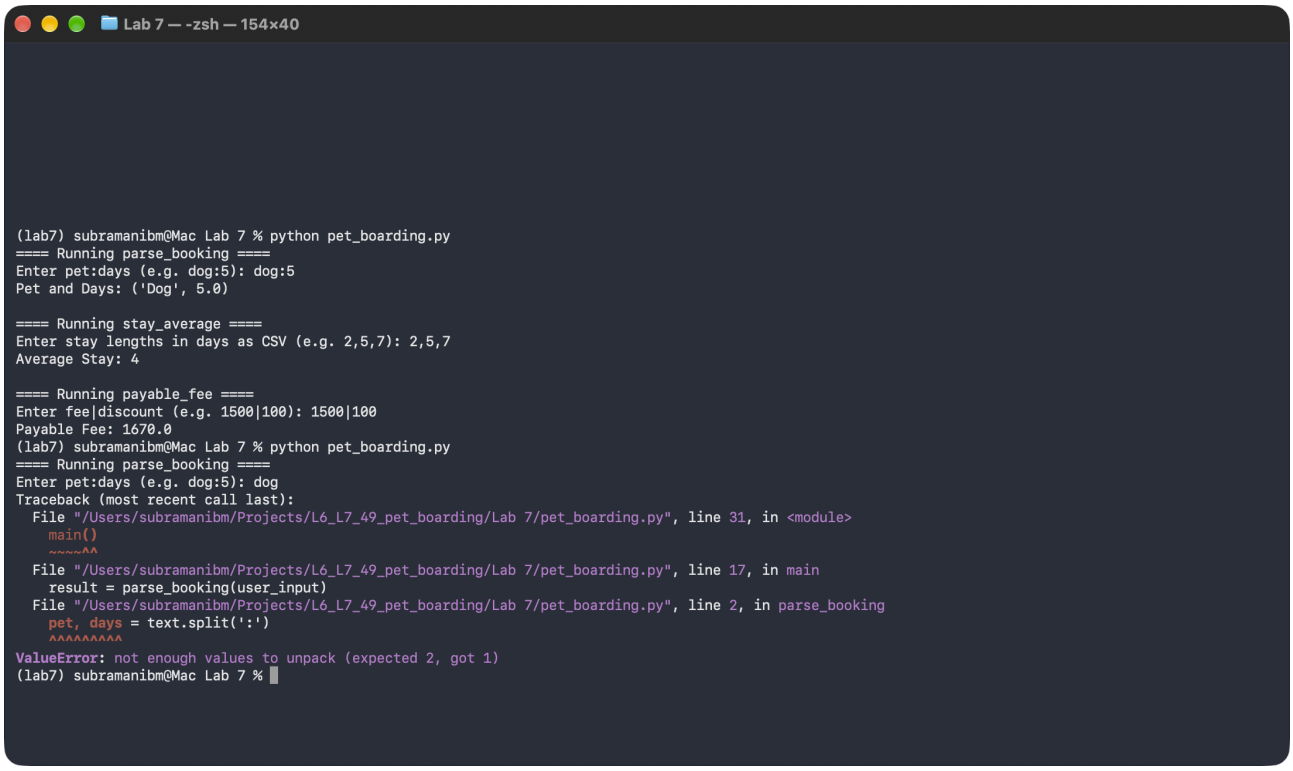


Figure 1 — Buggy pet_boarding.py: valid run, then a crash on input 'dog'

Step 3: Complete test_pet_boarding.py using Hypothesis

The three starter tests were completed and six more were added so each function is fuzzed in three ways. Each test runs 300 generated examples. The contract is: **never raise; return None for invalid input; any value returned must satisfy the function's invariants**.

Test	Strategy	Property checked
test_parse_booking_no_crash	st.text() | st.none()	No exception; result is None or (non-empty name, finite days > 0)
test_parse_booking_structured	pet text + float-as-text (nan, inf, negatives) or random text	Same invariants on inputs shaped like pet:days
test_parse_booking_accepts_valid_input	known pets, floats 0.5–365	Valid bookings are still parsed correctly
test_stay_average_no_crash	st.text() | st.none()	No exception; result is None or a positive int
test_stay_average_structured	lists of integers / random text joined by commas	Same invariants on CSV-shaped inputs
test_stay_average_within_bounds	lists of ints 1–10000	Result equals floor mean and lies between min and max
test_payable_fee_no_crash	st.text() | st.none()	No exception; result is None or a finite float ≥ 0
test_payable_fee_structured	float-as-text or random text on both sides of |	Same invariants on fee|discount inputs
test_payable_fee_matches_formula	floats 0–1e6	Result = round(max(fee×1.18 – discount, 0), 2)

Step 4: Run the tests against the buggy code

Command: `python -m pytest test_pet_boarding.py`. Result: **7 failed, 2 passed**. Hypothesis found 17 distinct failures and shrank each one to a minimal input.

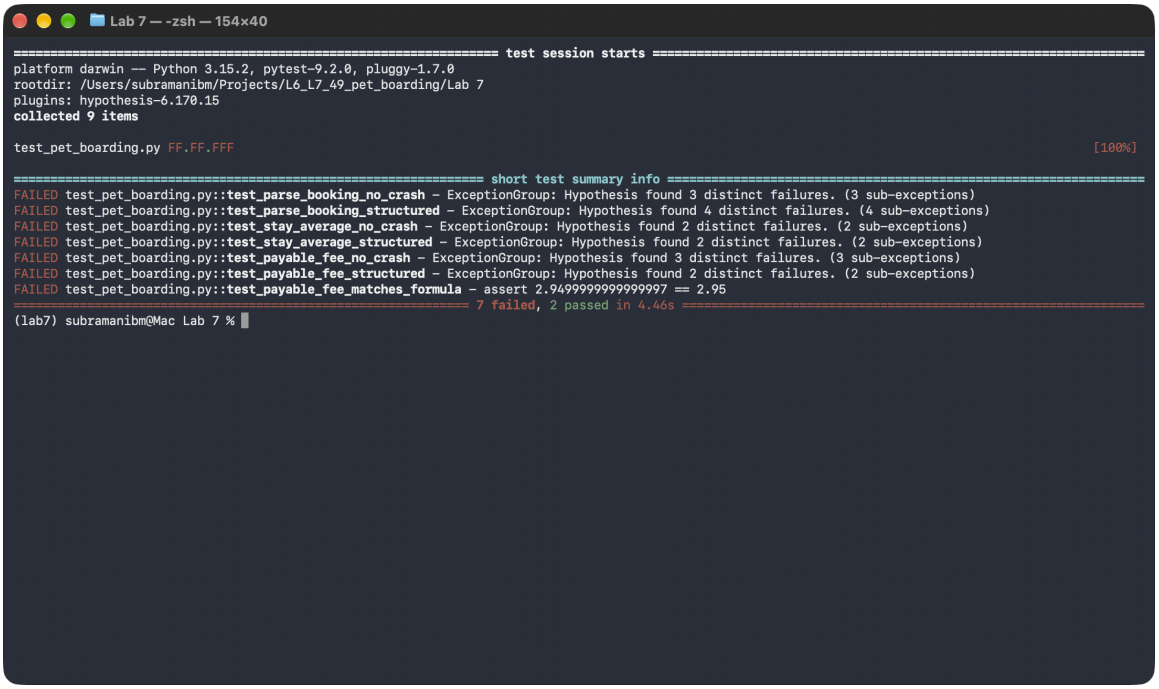


Figure 2 — Fuzz tests failing on the buggy code (summary)

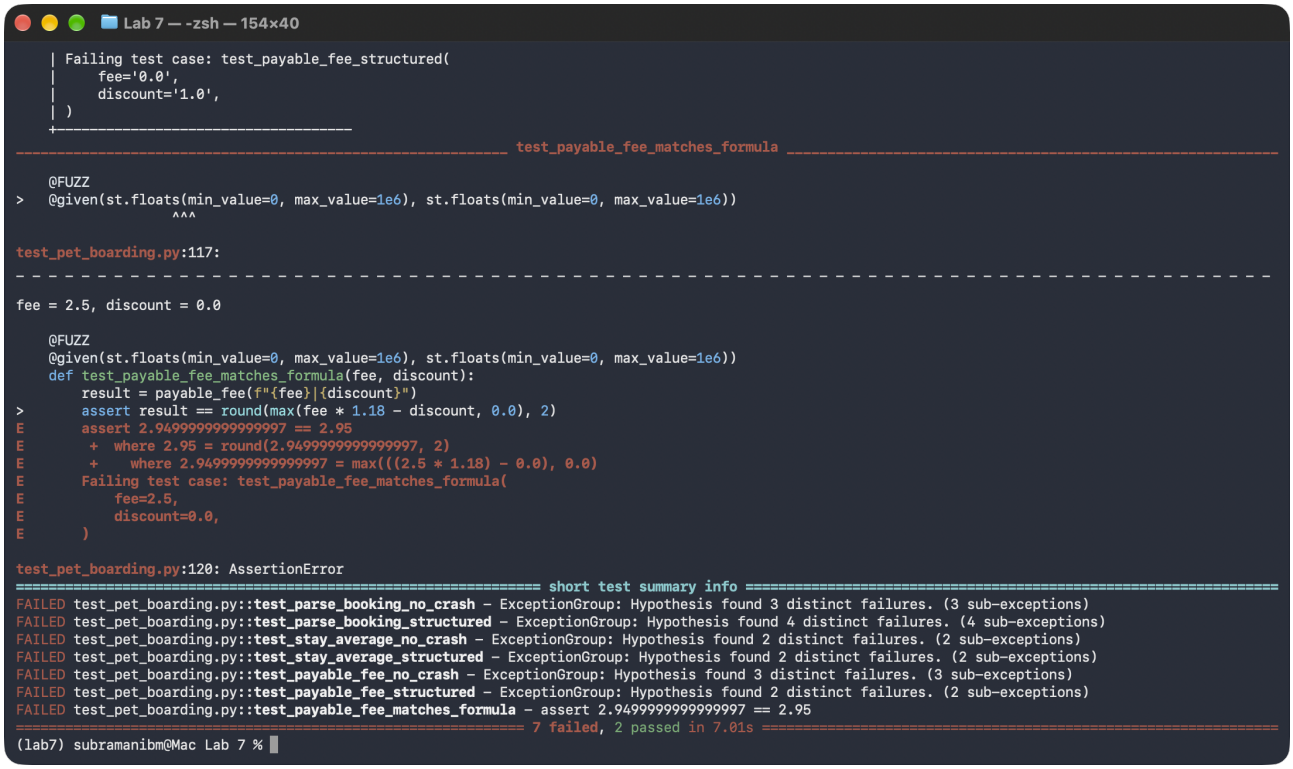


Figure 3 — Failure detail: shrunk failing examples reported by Hypothesis

Bugs found by fuzzing

Minimal inputs come from Hypothesis' shrinking, or from direct checks of the original code.

#	Function	Failing input	Original behaviour	Fix
1	all three	None	AttributeError: 'NoneType' has no attribute 'split'	Return None unless input is a str
2	parse_booking	'' , 'a:b:c'	ValueError: wrong number of values to unpack	Require exactly one ':'
3	parse_booking	':' , 'cat:abc'	ValueError: could not convert string to float	Safe number conversion (try/except)
4	parse_booking	':0.0' , ' :5'	Accepts an empty pet name	Pet name must be non-empty after strip
5	parse_booking	'cat:nan' , 'cat:-3' , '0:0.0'	Returns nan / negative / zero days	Days must be finite and > 0
6	stay_average	'' , '1,,2'	ValueError: invalid literal for int()	Reject empty or non-integer fields
7	stay_average	'0' , '-3,-5'	Returns 0 or a negative average (-4)	Every stay must be a positive integer
8	payable_fee	'' , 'a|b|c'	ValueError: wrong number of values to unpack	Require exactly one '|'
9	payable_fee	'|' , '0.0|'	ValueError: could not convert string to float	Safe number conversion
10	payable_fee	'nan|0' , 'inf|inf' , '1e309|0'	Returns nan or inf	Reject non-finite values and overflowed totals
11	payable_fee	'100|-500'	Negative discount raises the fee (618.0)	Fee and discount must be ≥ 0
12	payable_fee	'0.0|1.0' , '2.5|0'	Returns int 0; float noise 2.9499999999999997	Always return a float rounded to 2 decimals
13	main	closed stdin / Ctrl+C	EOFError / KeyboardInterrupt traceback	Caught; prints 'Invalid input' for bad values

Step 5: Fix the application and run it

Every function now checks its input type and format, converts numbers safely through a shared `_to_number()` helper that rejects nan and inf, enforces range rules, and returns `None` instead of raising. `main()` prints *Invalid input* for rejected values. Valid input gives the same results as before.

```
def parse_booking(text):
    if not isinstance(text, str):
        return None
    parts = text.split(':')
    if len(parts) != 2:
        return None
    pet = parts[0].strip()
    if not pet:
        return None
    days = _to_number(parts[1])          # None for non-numeric, nan, inf
    if days is None or days <= 0:
        return None
    return pet.title(), days
```

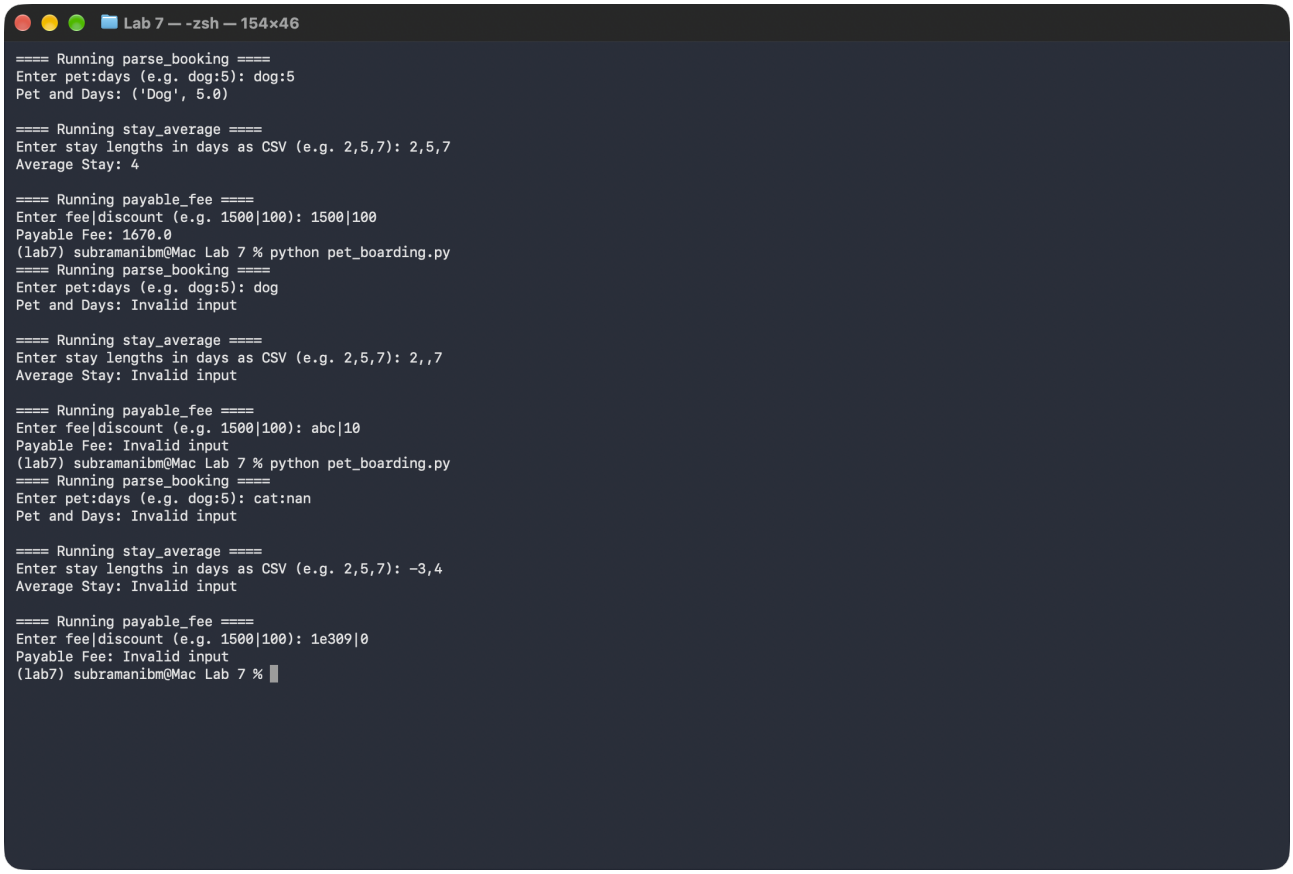


Figure 4 — Fixed pet_boarding.py: valid input still works; bad input is rejected

Step 6: Re-run the tests against the fixed code

Command: `python -m pytest test_pet_boarding.py`. Result: **9 passed** (2,700 generated inputs, no failures).

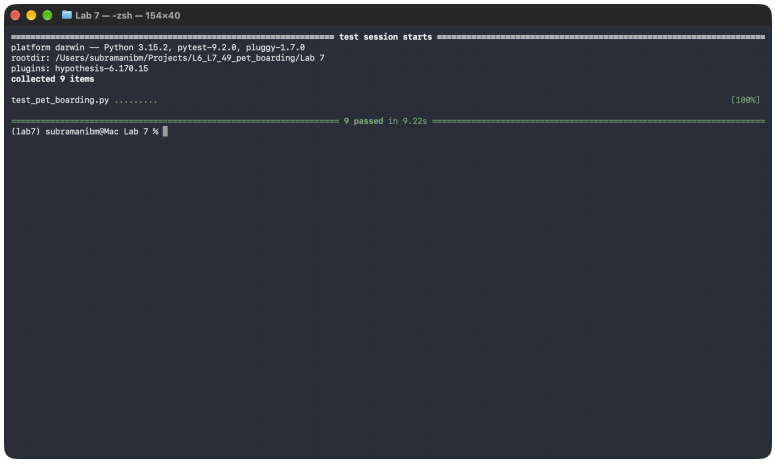


Figure 5 — All 9 fuzz tests pass on the fixed code

Conclusion

The three functions only worked on the happy path. Hypothesis found crashes on empty, None and malformed input within seconds. It also found silent wrong answers (nan days, negative averages, inf fees, a negative discount increasing the fee) that the obvious unit tests would have missed. After the fixes, all 9 properties hold across 2,700 random inputs, and valid bookings give the same results as the original code.